Computer Architectures
Exam of 18/02/2026

B2

Please read the following notes before proceeding

1) It is recommended to save the project on the C drive during development. Once you have completed your work, you must save the project inside the exam folder on the Z drive (the same folder where you found this document). **Projects saved elsewhere will not be collected and will be permanently lost.**
2) The assembly subroutine developed in the first exercise must comply with the ARM Architecture Procedure Call Standard (AAPCS) standard (in terms of parameter passing, returned value, callee-saved registers).
3) If you prefer, you can begin with the second exercise. Since it requires calling the assembly subroutine from the first exercise, you can create a stub in the assembly file to proceed. For example:

```
subroutineName    PROC
      ; to do: add code
      MOV r0, #10       ; example of a return value
      BX LR
      ENDP
```

4) To earn points on all exercises, it is recommended that you write at least some code for each.
5) **The final project must run on the actual board (e.g., button debouncing is required).**

Special characters with Italian keyboards

- Square brackets [ ]
  - Left bracket: ATL GR + [
  - Right bracket: ALT GR +]
- Curly brackets { }
  - Left bracket: LSHIFT + ALT GR + [
  - Right bracket: LSHIFT + ALT GR +]
- Hash # : ALT GR + à

Bitwise operations in C

Left shift by N bits = A<<N
Right shift by N bits = A>>N

A XOR B = A^B
A AND B = A&B (&& is the logical AND)
A OR B = A|B (|| is the logical OR)

Computer Architectures
Exam of 18/02/2026

B2

Question 1 (8 points)

The Hofstadter–Conway $10,000 sequence is defined as follows:
a(1) = a(2) = 1
a(n) = a(a(n - 1)) + a(n - a(n - 1)), for $n > 2$

The first terms of the sequence are:
1, 1, 2, 2, 3, 4, 4, 4, 5, 6, 7, 7, 8, 8, 8, 8, 9, 10, 11, 12, 12, 13, 14, 14, 15, 15, 15, 16, 16, 16, 16, 16, 17, 18, 19, 20, 21, 21, 22, 23, 24, 24, 25, 26, 26, 27, 27, 27, 28, 29, 29, 30, 30, 30, 31, 31, 31, 31, 32, 32, 32, 32, 32, 32, 33, 34, 35, 36, 37, 38, 38, 39, 40, 41, 42

Write the `HofstadterConway` subroutine in ARM assembly language. You must follow this prototype:

unsigned int *HofstadterConway(****unsigned int* v, int*** *dim****);***

The subroutine receives:
- the address of an uninitialized read-write data area
- the number of elements *dim*.

The subroutine must fill the data area with the first *dim* elements of the Hofstadter–Conway $10,000 sequence. Each element must be stored as a word (you must reserve sufficient space when allocating the read-write data area).
The subroutine must return the maximum value among the first *dim* elements of the Hofstadter–Conway $10,000 sequence.

Suggestion. Avoid a recursive implementation of the sequence. Instead, store each term in the data area, starting with a(1) and a(2). When computing a(n), access the data area to retrieve the previously computed elements a(n-1), a(n-2), etc.

Question 2 (10 points)

In this part, you need 3 timers, referred to in the text as timer *A*, timer *B*, and timer *C*. You are free to choose which timer to use as timer *A*. For timer *B* and *C*, it is suggested to use two standard Capture/Compare timers.
Some actions depend on whether a timer is running. You can use a flag or check the appropriate timer register to determine whether a timer is active.
Add the following functionalities to your project. The goal of this exercise is to generate a musical representation of the Hofstadter–Conway $10,000 sequence by mapping its values to sound frequency and duration.

1) In the `main` function, call the `HofstadterConway` subroutine passing an uninitialized integer array *v* of size 1000. As specified in the first question, the subroutine fills the array and returns a_{max}.
Initialize the D/A Converter so that its output is connected to the input of the external speaker (pin P0.26).
Finally, start timer *A* with a period of 50 ms.

2) When timer *A* generates an interrupt, check whether timer *B* and timer *C* are running. If either of them is running, the interrupt handler of timer *A* must terminate immediately.
Otherwise increment an index *i*. This index is used to access the *i*-th element of the Hofstadter–Conway $10,000 sequence. For example, at the first interrupt, you access the value a(1), since timer

B and timer *C* are initially stopped. Note that a(1) is stored in *v*[0], a(2) in *v*[1], and in general a(*n*) is stored in *v*[*n*−1]. When *i* reaches 1000, you access the value a(1000) and then stop the timer *A*.

Each time you increment the index *i*, compute two thresholds according to the following formula:

$$threshold = \frac{1}{k}\left(P_{max} - \frac{a(i) * (P_{max} - P_{min})}{a_{max}}\right)$$

The formula computes a floating-point value, but each threshold must be cast to an unsigned integer.
For the first threshold, use P_{max} = 5351, P_{min} = 1062, k = 1.
For the second threshold, use P_{max} = 40,000,000, P_{min} = 625,000, k = 5. Here, to avoid overflow due to intermediate multiplications, carefully choose the order of evaluation: it is recommended to compute $((P_{max} - P_{min}) / a_{max}) \cdot a(i)$.
Then, start timer *B* with the first threshold, so that when its timer counter reaches the threshold, an interrupt is generated and the timer counter is **reset**.
Start timer *C* with the second threshold, so that when its timer counter reaches the threshold, an interrupt is generated and the timer counter is **stopped and reset**.

3) Define the following array, which contains samples of a sinusoidal waveform:
uint16_t *SinTable*[45] = {410, 467, 523, 576, 627, 673, 714, 749, 778, 799, 813, 819, 817, 807, 789, 764, 732, 694, 650, 602, 550, 495, 438, 381, 324, 270, 217, 169, 125, 87, 55, 30, 12, 2, 0, 6, 20, 41, 70, 105, 146, 193, 243, 297, 353};

When timer *B* generates an interrupt, increment an index *j*. This index is used to access the *j*-th element of *SinTable*. Use that element to feed the DAC.
If the index *j* reaches 45, you must reset it (without stopping the timer B).

4) When timer *C* generates an interrupt, stop timer *B*. In other words, timer *C* determines the duration of the note played by timer *B*.